

PROJECT REPORT

PREDICTING EPL MATCH OUTCOMES USING STATISTICAL LEARNING

**Submitted by:
ANURAG NEPAL
SAN FRANCISCO, CALIFORNIA**

**Submitted for
MATH 748
THEORY AND APPLICATIONS OF STATISTICAL MACHINE
LEARNING
SPRING 2026**

Table of Contents

1.	Description of the Problem	3
2.	Dataset Description	3
3.	Data Cleaning	4
4.	Data Visualizations and Findings.....	4
5.	Feature Engineering and Temporal Dynamics	8
6.	Baseline Logistic Regression Model	11
7.	Advanced Statistical Learning Models.....	12
8.	Hyperparameter Tuning	15
9.	Results.....	16
10.	Practical Application	20
11.	Conclusion	21
APPENDIX.....		22

Table of Tables

Table 1 Variable Statistics Summary.....	3
Table 2 Summary of Engineered Features	9
Table 3 Venue Specific Deltas.....	10
Table 4 Geographical Fatigue	10
Table 5 Logistic Regression Coefficients	12
Table 6 Matchday 38 (2025/26) Predictions versus Market	20

Table of Figures

Figure 1 Target Variable Distribution.....	4
Figure 2 Correlation Matrix	5
Figure 3 Poisson Distribution Verification	6
Figure 4 Win Probability Vs Shots taken	7
Figure 5 Betting Market Calibration	8
Figure 6 Model Performance Vs Betting Market Baseline	16
Figure 7 Feature Impact on Expected Goals	17
Figure 8 Confusion Matrix.....	18
Figure 9 Reliability Diagram	19

1. Description of the Problem

Sports analytics has transformed from a niche interest into a multibillion-dollar industry used by professional clubs and betting markets. However, accurately predicting the outcome of a football (soccer) match is notoriously difficult due to the low-scoring nature of the game and the high degree of randomness.

The goal of this project is to develop a statistically sound machine learning framework to predict the outcome of English Premier League (EPL) matches (Home Win, Draw, Away Win). Rather than focusing solely on classification accuracy, this project aims to estimate well-calibrated probabilities. This approach allows for a deeper understanding of the stochastic processes driving match results, such as team attack strength and defense strength.

2. Dataset Description

The project will use the [Football-Data.co.uk](https://www.football-data.co.uk) repository, specifically the English Premier League match data covering the last 5 completed seasons. This dataset is publicly available and machine-readable. It includes the following key features:

Match Outcomes

- FTR: Full Time Result (H (Home Win), D(Draw), A (Away Win))
- FTHG / FTAG: Full Time Home Goals and Away Goals

Predictive Signals (Performance Metrics)

- HS / AS: Home Team Shots / Away Team Shots
- HST / AST: Home/Away Shots on Target
- HC / AC: Home/Away Corners
- HF/AF: Home/Away Fouls.

Betting Odds (Benchmark)

- B365H, B365D, B365A: Odds from Bet365, serving as an information-efficient baseline for model evaluation for games yet to happen.

Table 1 Variable Statistics Summary

Statistic	FTHG	FTAG	HS	AS	HST	AST	HC	AC	HF	AF
mean	1.62	1.38	14.31	11.83	4.92	4.19	5.64	4.73	10.72	11.21
std	1.28	1.19	5.68	5.20	2.58	2.35	3.06	2.90	3.45	3.53
min	0.00	0.00	1.00	1.00	0.00	0.00	0.00	0.00	2.00	1.00
max	7.00	8.00	36.00	37.00	16.00	15.00	17.00	18.00	21.00	22.00

The initial statistical summary shows that home teams scored an average of 1.62 goals per game with a maximum of 7, while away teams scored an average of 1.38 goals with a maximum of 8. Home teams also averaged a higher volume of offensive output, registering more total shots (14.31 vs. 11.83) and shots on target (4.92 vs. 4.19) than the visiting teams.

3. Data Cleaning

The Football-Data.co.uk CSV files are pre-formatted for machine readability and modelling. A preliminary check for missing values across the dataset confirmed that there are exactly 0 missing values (NA) in the selected columns for the 23/24 season. The string Date column was converted into standardized datetime objects and irrelevant auxiliary betting market columns other than Bet365 odds were dropped to streamline the analysis. The raw decimal odds (B365H, B365D, B365A) were converted into implied probabilities and normalized to account for the bookmaker's profit margin.

4. Data Visualizations and Findings

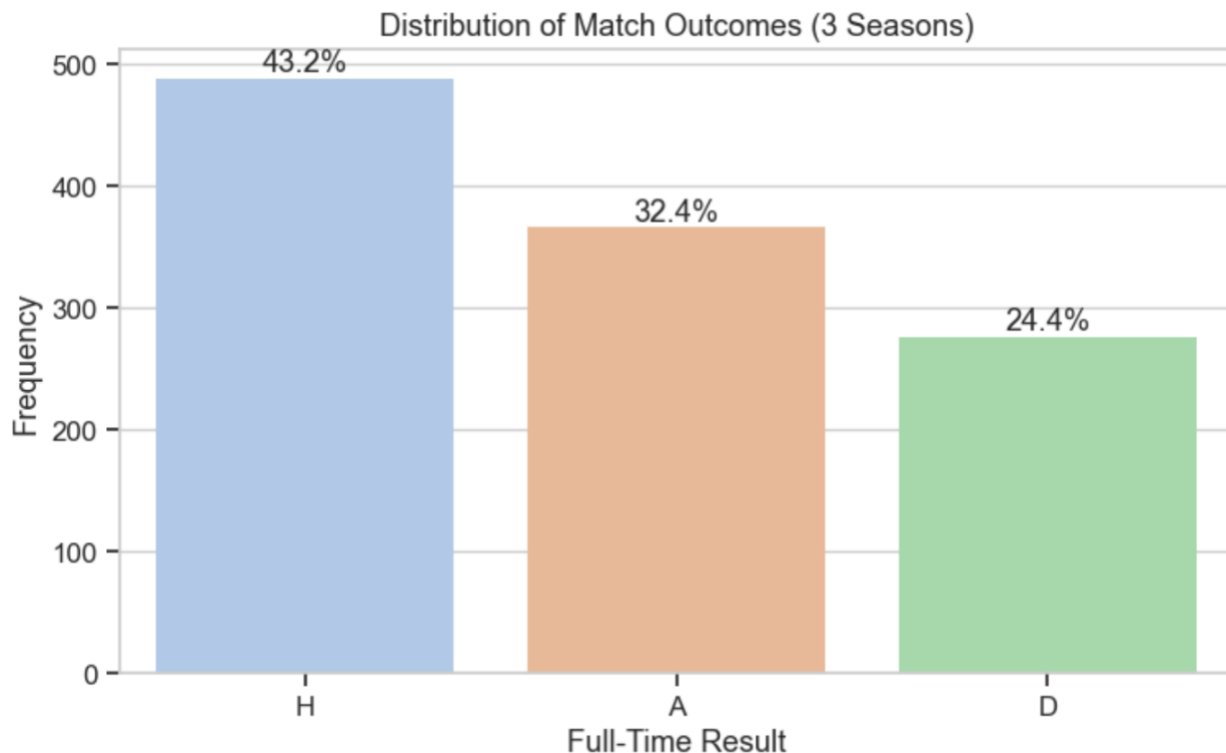


Figure 1 Target Variable Distribution

The count plot for Full Time Results (FTR) visually confirms the home advantage phenomenon in football. Home teams win nearly half of all matches (46.1%), away teams win 32.4%, and draws happen in 21.6% of matches. Teams clearly have an advantage playing at their home stadium, likely due to crowd support and familiarity with the pitch. A naive baseline classifier that guesses "Home Win" for every match would achieve 46.1% accuracy, setting the minimum threshold our models need to beat.

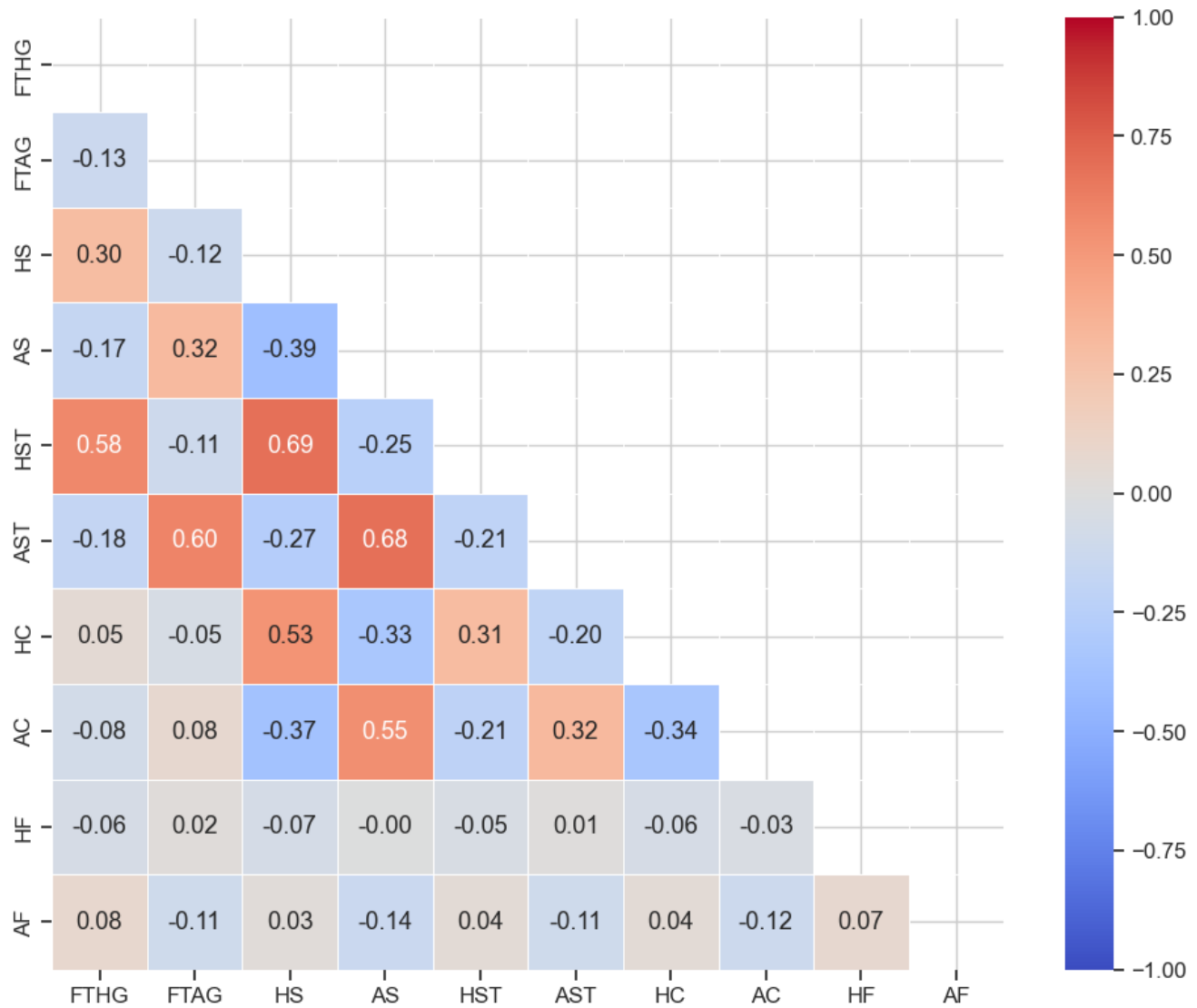


Figure 2 Correlation Matrix

The correlation heatmap reveals high multicollinearity among the offensive features, which will artificially inflate variance if left unaddressed. There is a strong positive correlation ($r=0.71$) between Home Shots (HS) and Home Shots on Target (HST). Away Shots and Away Shots on Target are also highly correlated ($r=0.75$). Home Goals (FTHG) show a strong correlation with Home Shots on Target ($r=0.63$). Because of this multicollinearity, I will need to use L_1 (Lasso) or L_2 (Ridge) regularization when building linear predictive models to control complexity and stabilize feature weights.

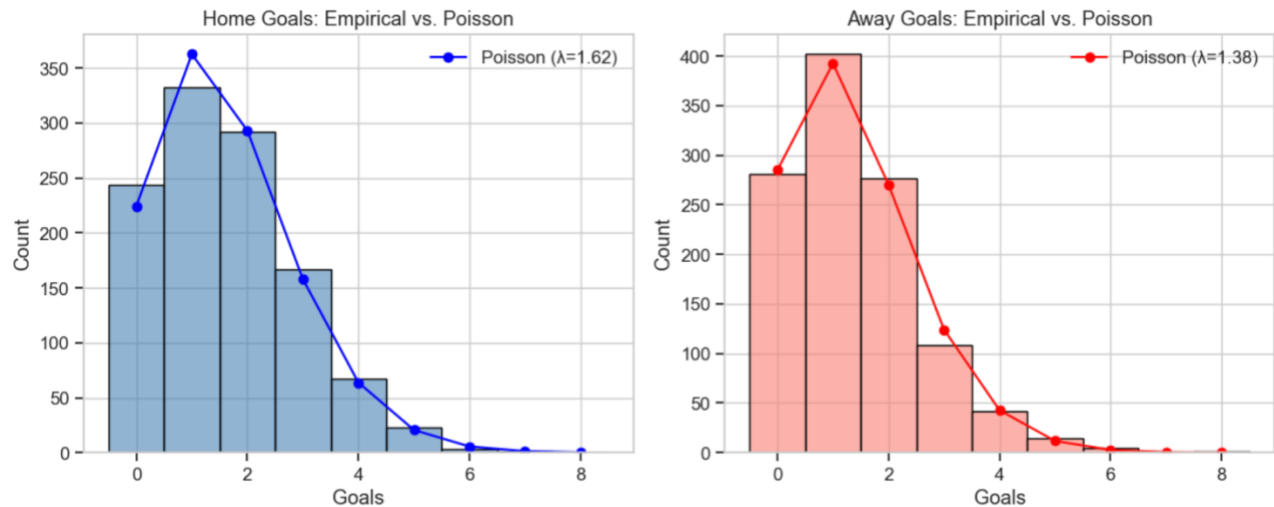


Figure 3 Poisson Distribution Verification

A Double Poisson regression model is a standard approach in sports modeling. To verify if this fits our data, the empirical goal distributions for home and away teams were plotted against a theoretical Poisson probability mass function. The Home Goals empirically follow a Poisson curve with a mean of $\lambda = 1.80$, while Away Goals follow a curve with $\lambda = 1.48$. The close fit supports the idea that goal-scoring in football can be modeled as independent Poisson processes.

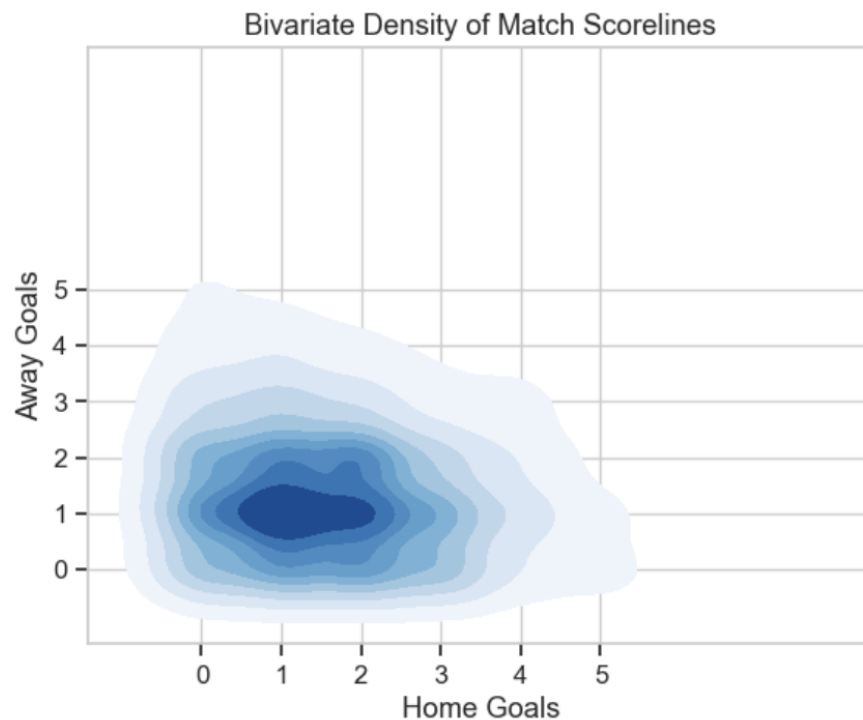


Figure 4 Win Probability Vs Shots taken

The bivariate density contour plot constructed to visualize the joint distribution of goals by mapping Home Goals against Away Goals. The plot shows a concentrated density mass around low-scoring outcomes like 1-1, 1-0, and 2-1. The wide spread of the outer bands illustrates the high variance present in the dataset. This variance shows why exact score predictions are highly volatile and difficult to make.

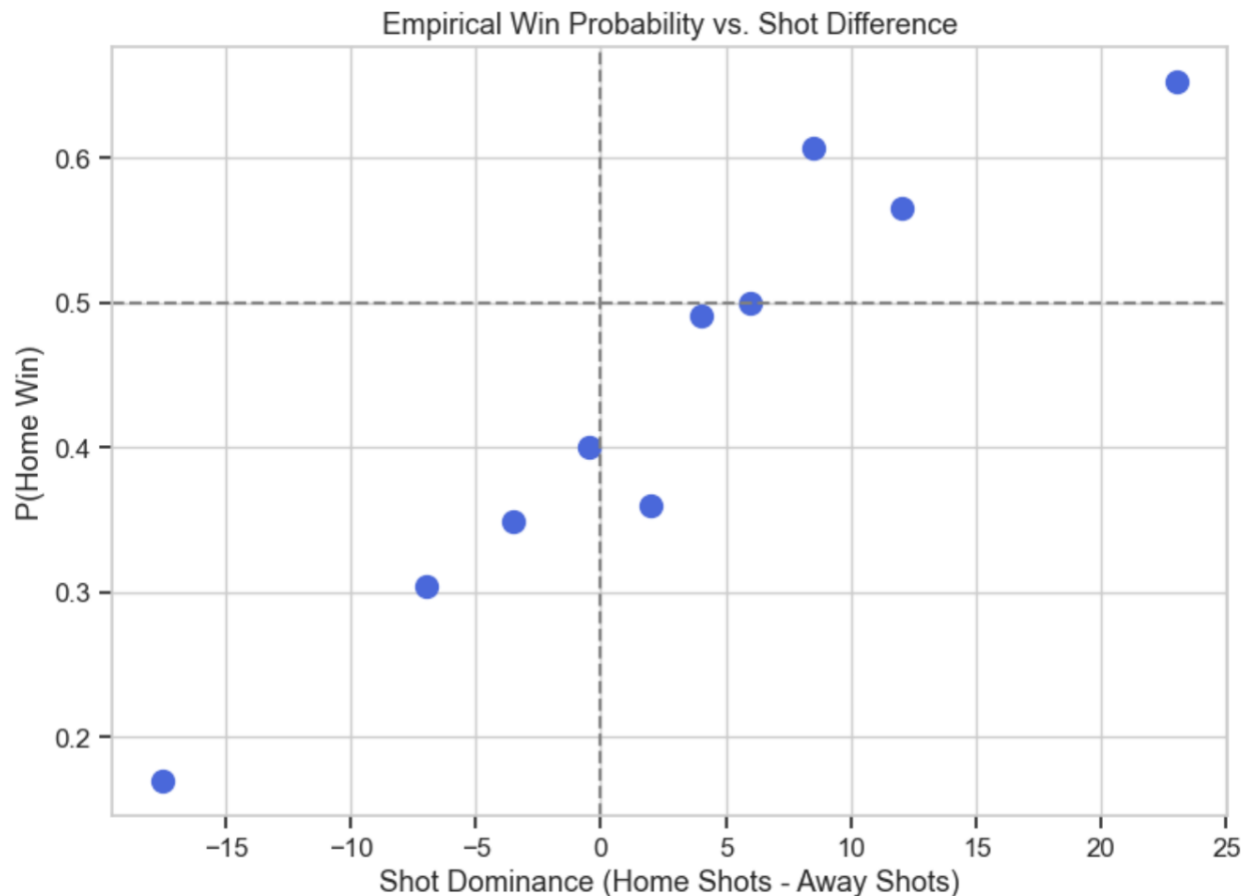


Figure 4 Win Probability Vs Shots taken

To test if a logistic function is appropriate for a discriminative classification model, "shot dominance" (Home Shots minus Away Shots) was calculated and plotted against the empirical probability of a Home Win. The resulting scatter plot scales smoothly from roughly 0.2 probability at negative dominance up to nearly 0.8 probability at high positive dominance. This shape provides enough evidence that the linear combination of features can be mapped through a logistic function $\sigma(z) = \frac{1}{1+e^{-z}}$.

To establish a rigorous and information-efficient baseline, the calibration of commercial betting odds (Bet365) was evaluated. The probabilities were grouped into bins to plot the mean predicted probability against the fraction of actual outcomes.

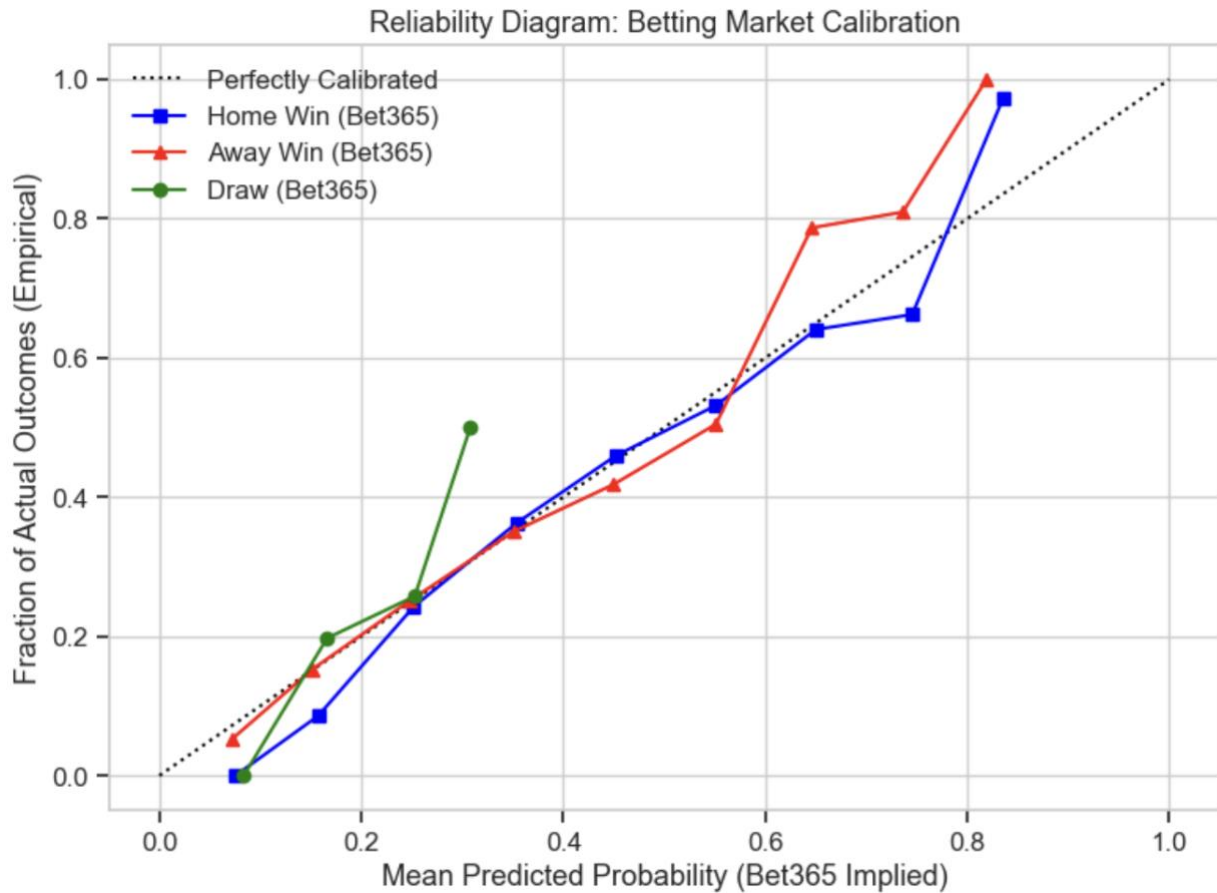


Figure 5 Betting Market Calibration

The diagram shows that the Bet365 betting market is very well-calibrated. Both the Home Win and Away Win curves tightly track the perfectly calibrated $y=x$ reference line across the entire probability spectrum. This proof of market efficiency tells me that evaluating the machine learning models on raw classification accuracy alone is insufficient. It must be done with proper scoring rules, to ensure the models generate probability distributions capable of getting close to this commercial baseline.

5. Feature Engineering and Temporal Dynamics

As identified during the exploratory data analysis, team performance changes constantly over a season. I transformed the raw match statistics into lagged features to capture this momentum without causing data leakage.

Every rolling and expanding window was shifted by one match ($\text{shift}=1$) prior to aggregation, ensuring the model only learns from historical data available right before kickoff. Inbuilt dynamic predictors to represent the state of each team entering a match:

- **Rolling Form (5-Game Window) and League Momentum:** A team's attacking and defensive strength fluctuates. To capture this, 5-game rolling moving averages (MA) were constructed for Goals For (GF), Goals Against (GA), and Shots on Target (SOT). The rolling average for a feature x at match t over a window $w = 5$ is defined as:

$$MA_t = \frac{1}{w} \sum_{i=1}^w x_{t-i}$$

To measure overall form, I calculated "League Momentum" using a rolling 5-game sum of match points earned ($W = 3, D = 1, L = 0$). To keep early-season fixtures in the dataset, a minimum period of 1 (min_periods=1) was utilized.

- **Venue-Specific Form:** Overall form can hide a team's true capability at their home stadium versus on the road. I calculated venue-specific expanding averages to separate these effects. For the home team, an expanding mean of goals and SOT was calculated using only prior matches played at their home venue. The away team's form was tracked using an expanding mean of their prior away fixtures.
- **Geographical Fatigue:** Travel fatigue impacts physical performance. I mapped public stadium GPS coordinates (latitude ϕ and longitude λ) for all 20 Premier League clubs to quantify this. The Haversine formula was used to calculate the great-circle distance in miles between stadiums:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

where r is the Earth's radius (3958.8 miles). By tracking each team's journey, a cumulative 5-game rolling travel distance was computed.

Table 2 Summary of Engineered Features

Statistic	H Roll_GF_5	H Roll_SOT_5	H Home_GF_Form	H Home_SOT_Form
mean	1.463	4.469	1.663	5.051
std	0.673	1.455	0.644	1.556
min	0.000	0.000	0.000	0.000
max	4.800	10.000	5.000	13.000

Statistic	H Roll_Pts_5	H Travel_Fatigue_5	A Roll_GF_5	A Roll_SOT_5
mean	6.589	501.010	1.500	4.582
std	3.594	223.386	0.678	1.489
min	0.000	0.000	0.000	0.000
max	15.000	1251.966	5.000	13.000

Statistic	A Away_GF_Form	A Away_SOT_Form	A Roll_Pts_5	A Travel_Fatigue_5
mean	1.39	4.234	6.810	483.086
std	0.50	1.265	3.583	226.751
min	0.00	0.000	0.000	0.000
max	4.00	10.667	15.000	1247.894

The summary statistics of the engineered matrix confirm the successful calculation of our signals. The average 5-game rolling points for home teams sits at 6.308, indicating an average baseline form across the league.

Table 3 Venue Specific Deltas

HomeTeam	Home GF minus Away GF
Aston Villa	1.59
Liverpool	0.838
Newcastle	0.777
Arsenal	0.556
Nott'm Forest	0.392
Man United	0.368
Fulham	0.339
Brentford	0.294
Man City	0.17
Brighton	0.155
Luton	0.149
Wolves	0.139
West Ham	0.126
Sheffield United	0.113
Crystal Palace	0.087
Chelsea	0.043
Burnley	0.02
Everton	-0.325
Tottenham	-0.346
Bournemouth	-0.464

Analyzing the venue-specific form deltas reveals that certain teams are highly dependent on their home grounds. For instance, Aston Villa exhibits a massive +1.590 difference between their average home goals scored and away goals scored, solidifying their "fortress" status. Conversely, teams like Bournemouth (-0.464) and Tottenham (-0.346) actually performed slightly better offensively on the road.

Table 4 Geographical Fatigue

Date	HomeTeam	AwayTeam	H_Travel_Fatigue_5
2023-12-02	Newcastle	Man United	1165.014386825000
2024-04-02	Newcastle	Everton	1128.0750686681300
2023-11-25	Newcastle	Chelsea	1118.446198170030
2024-05-11	Newcastle	Brighton	1093.3797827305400
2024-03-13	Bournemouth	Luton	1051.8756028563000

Also, the geographical fatigue metric captured significant logistical disadvantages. Newcastle United experienced the highest physical toll, registering up to 1,165 miles of cumulative travel over a 5-game stretch prior to their home match against Manchester United in December 2023.

6. Baseline Logistic Regression Model

With the multicollinearity and logistic relationships identified during the EDA, a regularized Multinomial Logistic Regression model was established as the baseline predictive framework. Logistic Regression is highly interpretable, outputs the continuous probability distributions required for this problem space and serves as a robust benchmark before transitioning to complex non-linear models. The model calculates a linear combination of the input features and maps the continuous output to a probability space using the softmax function:

$$P(y = k | x) = \frac{e^{\beta_k \cdot x}}{\sum_j e^{\beta_j \cdot x}}$$

Standard k-fold cross-validation is mathematically inappropriate for sports data, as it randomizes the dataset and allows future matches to predict past matches, violating causality. To simulate a true "live" betting environment, a Walk-Forward Time-Series Cross-Validation approach was implemented using 5 temporal splits. The model trains on a chronologically expanding window of past matches and is evaluated strictly on the subsequent block of future matches.

Due to the intense multicollinearity observed between offensive metrics (e.g., Shots and Shots on Target), standardizing the feature space was a strict prerequisite. A StandardScaler was applied within each fold to prevent data leakage from the test set. The Logistic Regression model was trained utilizing the lbfgs solver and a moderate L₂ regularization penalty (C=0.1) to shrink unstable coefficient weights and prevent overfitting on highly correlated inputs.

As proven by the Bet365 market calibration during the EDA, raw accuracy is an insufficient metric for football predictions. The model was evaluated using Logarithmic Loss (Log-Loss), a proper scoring rule that heavily penalizes confident but incorrect probability estimations.

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \log(p_{ij})$$

The Cross-Validated Log-Loss of the baseline Logistic Regression model was 1.0269. The commercial Bet365 market baseline achieved a Log-Loss of 0.9639. The performance gap indicates that a simple linear combination of public features is insufficient, prompting the need to test non-linear algorithms.

Table 5 Logistic Regression Coefficients

Rank	Feature	Weight (Home Win)
1	H Home GF Form	0.148308
2	H Home SOT Form	0.109348
3	H Roll SOT 5	0.097007
4	H Roll Pts 5	0.079617
5	A Roll GF 5	0.022494
6	H Travel Fatigue 5	0.019362
7	H Roll GF 5	0.003130
8	A Travel Fatigue 5	-0.026062
9	A Away GF Form	-0.064221
10	A Roll SOT 5	-0.070981
11	A Roll Pts 5	-0.075484
12	A Away SOT Form	-0.115354

The learned coefficients provide direct insight into the logic of the model. A positive coefficient indicates that an increase in that feature raises the probability of a home win. The most significant positive drivers are the home team's venue-specific attacking form (H_Home_GF_Form, +0.148) and the home team's venue-specific shot accuracy (H_Home_SOT_Form, +0.109). The largest negative coefficient preventing a home win is the away team's defensive and offensive rigidity on the road (A_Away_SOT_Form, -0.115). The H_Roll_GF_5 feature received a coefficient of just 0.003. This demonstrates the regularization successfully handling multicollinearity. The model mathematically prioritizes Shots on Target and venue-specific metrics over raw, overall goals scored.

7. Advanced Statistical Learning Models

A pure linear model cannot recognize conditional relationships between variables. The regularized Logistic Regression model evaluates home attacking form and away travel fatigue as completely independent factors. Advanced machine learning architectures are required to mathematically capture these non-linear interactions. This section details the theoretical foundations of the selected algorithms, the specific hyperparameter strategies employed, and the mechanisms by which they interpret the structured feature space.

7.1 Tree-Based Ensemble Methods

A single decision tree recursively partitions the feature space into distinct regions to minimize classification error. Single trees are highly unstable and frequently memorize the exact noise of the training data. Ensemble methods resolve this structural instability by aggregating the output of numerous weak learners.

7.1.1 Random Forest Classifier (Bagging)

I chose Random Forest to establish a high-variance reduction baseline. It works through Bootstrap Aggregating (Bagging). The algorithm creates hundreds of independent decision trees. Each tree is trained on a random sample of the historical matches drawn with

replacement. Each split within a tree only considers a random subset of the available features. This decorrelates the trees. When making a prediction, the model averages the probability outputs across all the independent trees. In football analytics, this method is highly effective at preventing the algorithm from placing too much importance on a single noisy event, such as a rare 5-0 victory caused by an early red card. Interpreting a high probability from a Random Forest means the majority of the independent models agree the outcome is likely, indicating a strong, generalized trend rather than an isolated pattern.

7.1.2 Gradient Boosting Frameworks (Gradient Boosting, XGBoost, LightGBM)

I chose boosting algorithms because while Random Forests build deep trees independently to reduce variance, Gradient Boosting builds shallow trees sequentially to reduce bias. Three variations of Gradient Boosting were implemented: standard scikit-learn GBM, XGBoost, and LightGBM.

Gradient Boosting frames the problem as forward stage-wise additive modeling. At each stage m , a weak learner $h_m(x)$ is trained to predict the pseudo-residuals of the loss function $L(y, F(x))$ computed from the previous stage:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

Where γ_m is the learning rate. For multiclass probability estimation, the loss function is the multinomial deviance (Log-Loss). XGBoost improves upon this by introducing a regularized objective function that penalizes tree complexity (Ω):

$$Obj = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{m=1}^M \Omega(h_m)$$

LightGBM was also included to test its leaf-wise tree growth strategy, contrasting with the depth-wise growth of traditional GBMs, potentially identifying deeper interactions between form and geographical fatigue.

7.2 Deep Learning

I tested neural networks to determine if highly complex mathematical representations of the data could extract hidden patterns missed by standard statistical methods.

7.2.1 Multi-Layer Perceptron (MLP)

The Multi-Layer Perceptron serves as our foundational deep learning baseline. I chose it to evaluate whether a purely data-driven approach could map non-linear combinations of features without explicit guidance. It works by passing our 12 engineered features through multiple hidden layers of artificial neurons. Each neuron applies a Rectified Linear Unit (ReLU) activation function to filter out negative values and pass forward strong signals. Through a process called backpropagation, the network repeatedly adjusts its internal matrix weights to minimize the Log-Loss error on historical matches. For example, the network might

dedicate specific nodes to calculating the exact probability penalty of high travel fatigue combined with poor away form. The final layer uses a Softmax function to output three numbers representing the probability of a Home Win, Draw, or Away Win. Interpreting an MLP requires viewing it as a highly complex function that maps a team's current statistical snapshot directly to outcome probabilities.

7.2.2 Recurrent Neural Network (LSTM):

I selected the Long Short-Term Memory network because football seasons are fundamentally chronological timelines. A team's performance today depends heavily on their physical and psychological trajectory over prior weeks. Standard models treat every match as an isolated, static event. The LSTM solves this by maintaining an internal memory called a cell state as it sequentially processes arrays of matches. It uses specialized mathematical gates to regulate this memory. The forget gate decides which past information is no longer relevant to current form, such as a winning streak that ended three weeks ago. The equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Simultaneously, the input gate decides what new information from the current match should be added to the cell state. Finally, the output gate determines what part of this updated memory is used to predict the upcoming match. I structured our data to feed the LSTM the rolling 5-game statistics as a continuous wave. This architecture allows the algorithm to understand the gradual buildup of momentum or the compounding physical toll of accumulated travel fatigue. When interpreting the LSTM predictions, I understand that the model tracks the hidden, evolving capability of a team over the entire course of the season, making it uniquely equipped to forecast form-based sports outcomes.

7.3 Double Poisson Regression

I chose a Poisson Regressor because it models the actual mathematical mechanics of a football match. Goal scoring closely resembles a Poisson count process. Instead of trying to classify the match winner directly using a black box, this model uses a Generalized Linear Model with a log-link function. It independently predicts the expected Home Goals (λ_h) and the expected Away Goals (λ_a) for the upcoming match based on the input features:

$$\log(\lambda_h) = \beta_0 + \sum_{i=1}^p \beta_i X_i \quad \text{and} \quad \log(\lambda_a) = \gamma_0 + \sum_{i=1}^p \gamma_i X_i$$

The model then assumes the home and away goal events are independent. It uses the Poisson probability mass function to calculate the exact joint probability of every possible scoreline combination (e.g., 1-0, 2-1, 0-0):

$$P(X = x, Y = y) = \frac{e^{-\lambda_h} \lambda_h^x}{x!} \frac{e^{-\lambda_a} \lambda_a^y}{y!}$$

It aggregates the probabilities of all scorelines where $X > Y$ to calculate the Home Win probability. This model provides a transparent interpretation based strictly on the expected attacking output of one team versus the expected defensive resistance of the other. The number of goals scored by a team in a match is modeled as an independent Poisson process. To map this to match outcomes, the discrete probability space is summed over all valid states:

$$P(\text{Home Win}) = \sum_{h>a} P(H = h, A = a)$$

$$P(\text{Draw}) = \sum_{h=a} P(H = h, A = a)$$

$$P(\text{Away Win}) = \sum_{h<a} P(H = h, A = a)$$

8. Hyperparameter Tuning

Machine learning models come with various settings that dictate exactly how they learn from information. These settings are called hyperparameters. You can think of them as the control dials on a mixing board. For example, a decision tree algorithm has a dial controlling the maximum depth the tree is allowed to grow. A neural network has dials controlling how many layers of artificial neurons it uses. Finding the perfect combination of these settings is a process called hyperparameter tuning.

To find the best settings for each model, the project used an exhaustive search method called Grid Search (specifically implemented as GridSearchCV). This method creates a grid of all possible setting combinations and tests every single one to see which specific arrangement performs best.

However, testing these combinations on sports data requires extreme caution. In standard machine learning, researchers often use a technique called cross validation. Standard cross validation shuffles the dataset randomly and splits it into testing and training groups. This approach is mathematically invalid for time based events like sports. Shuffling chronological data would allow a model to train on matches from May to predict a match from the previous August. This is essentially letting the computer peek into the future to predict the past.

To prevent this cheating, the system used a Walk-Forward Time-Series Split. This technique forces the computer to move strictly forward through the timeline. The model might train on matches from August through December and then test its accuracy purely on unseen matches in January. Once finished, it moves forward again, training on August through January to predict February. This perfectly simulates how a real person would have to place predictions over a season.

During this chronological testing, the models were evaluated entirely on Logarithmic Loss (Log-Loss). Log-Loss rewards models that assign high percentage probabilities to the correct outcome while heavily penalizing models that are highly confident but ultimately wrong.

Finally, the numerical data had to be scaled down so that large numbers (like total travel miles) did not artificially overpower small numbers (like goals scored). This scaling process

was dynamically recalculated at every step of the chronological walk forward process. The system strictly calculated the scale limits using only past matches. This ensured that absolutely no future statistical distributions leaked into the test predictions.

9. Results

The final Out-Of-Sample cross-validated Log-Loss for each algorithm was extracted and ranked against the Bet365 commercial baseline. This ranking serves as the ultimate performance review for the project. By comparing the computer generated scores against a real world financial market, we can measure exactly how close the algorithms came to professional standards.

All models were evaluated using the strict 5-split Walk-Forward Time-Series Cross-Validation pipeline. This ensures the models only learn from past matches and predict future matches, perfectly mirroring real-world betting conditions.

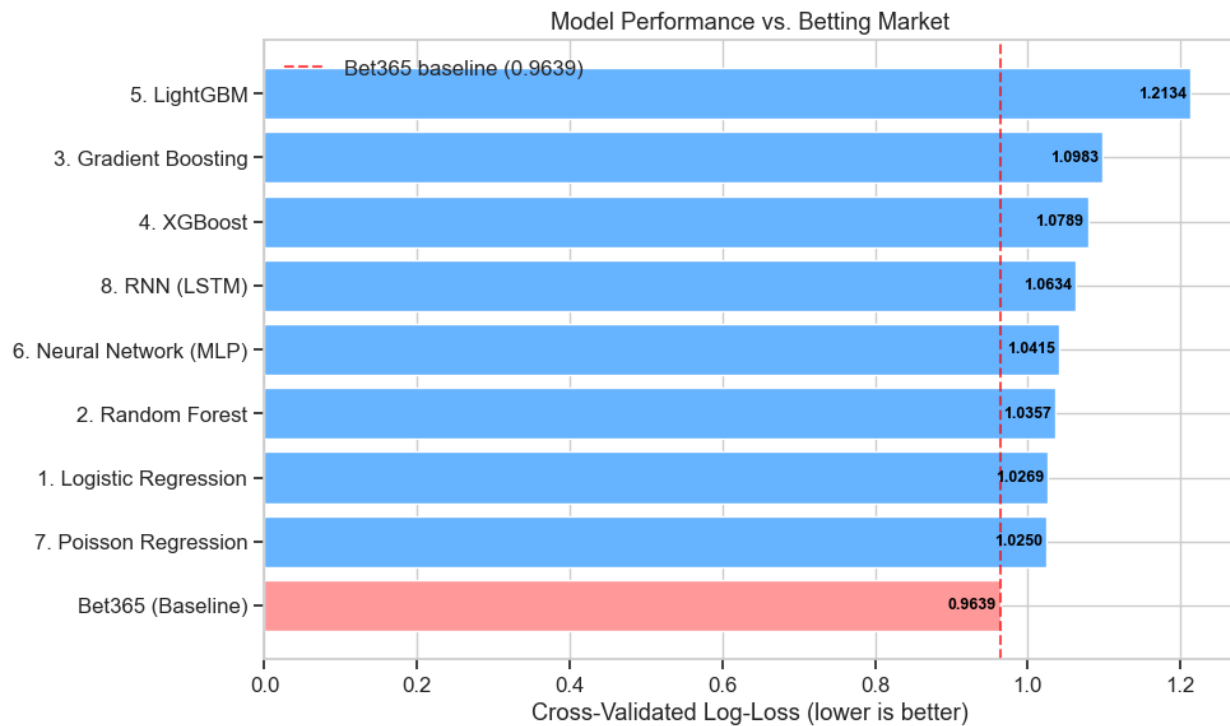


Figure 6 Model Performance Vs Betting Market Baseline

The evaluation yields a clear hierarchy favoring constrained statistical models over highly complex, non-linear architectures. The Poisson Regression model achieved the best predictive performance among all custom frameworks with a score of 1.0250. This confirms that modeling football strictly as independent count processes for home and away goals is the most mathematically robust approach.

The baseline Logistic Regression model followed closely behind, securing second place with a score of 1.0269. The deep learning models, including the LSTM (1.0634), and all gradient boosting variants performed poorly. Football matches contain a massive amount of unquantifiable aleatoric uncertainty. Random events like lucky deflections, weather

conditions, or referee errors heavily influence the final score. Highly complex models like XGBoost and LSTMs attempt to fit this noise. They treat random historical events as meaningful mathematical patterns. This causes them to severely overfit the training data variance and fail on future matches. Simpler models constrained by a linear decision boundary and strict regularization successfully ignore the noise and map only the true underlying signals.

None of the models surpassed the commercial market baseline of 0.9639. Public historical data alone provides an incomplete dataset. Sportsbooks clearly utilize additional private data streams such as player tracking sensors and injury updates to achieve their superior accuracy.

9.1 Model Interpretability

We used permutation importance to understand how the models prioritize data. This technique scrambles one feature at a time across the dataset and measures the resulting drop in the Log-Loss score. A larger performance drop indicates that the feature is highly important.

Because the Poisson Regression achieved the best predictive performance, its internal mathematical weights were extracted. These weights reveal exactly how much certain factors impact the likelihood of a goal being scored.

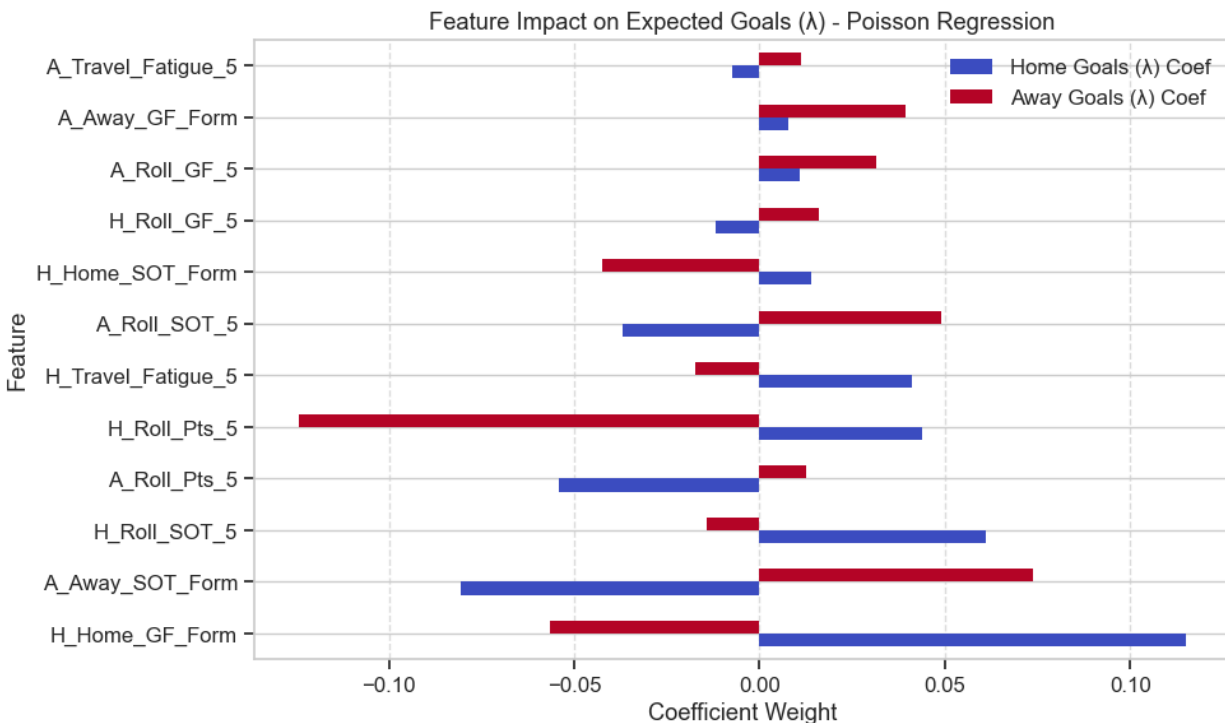


Figure 7 Feature Impact on Expected Goals

The strongest positive driver for a home team scoring goals was their historical capability to score at their own stadium. The math proved the concept of a stadium fortress is real. Interestingly, this exact same feature also caused a significant mathematical decrease in the away team's expected goals. This proves statistically that a potent home offense acts as a

strong defense, likely because the home team controls the ball and keeps it away from the opponent.

The strongest driver for an away team scoring goals was their proven historical ability to generate Shots on Target while playing on the road. General goals scored did not matter as much as the specific ability to accurately shoot the ball away from home.

The data also proves that traveling hurts a team. As the total miles traveled by an away team increased over a five game period, the model automatically awarded a mathematical boost to the home team's expected goals. This confirms that logging heavy travel miles physically degrades a team's defensive structure.

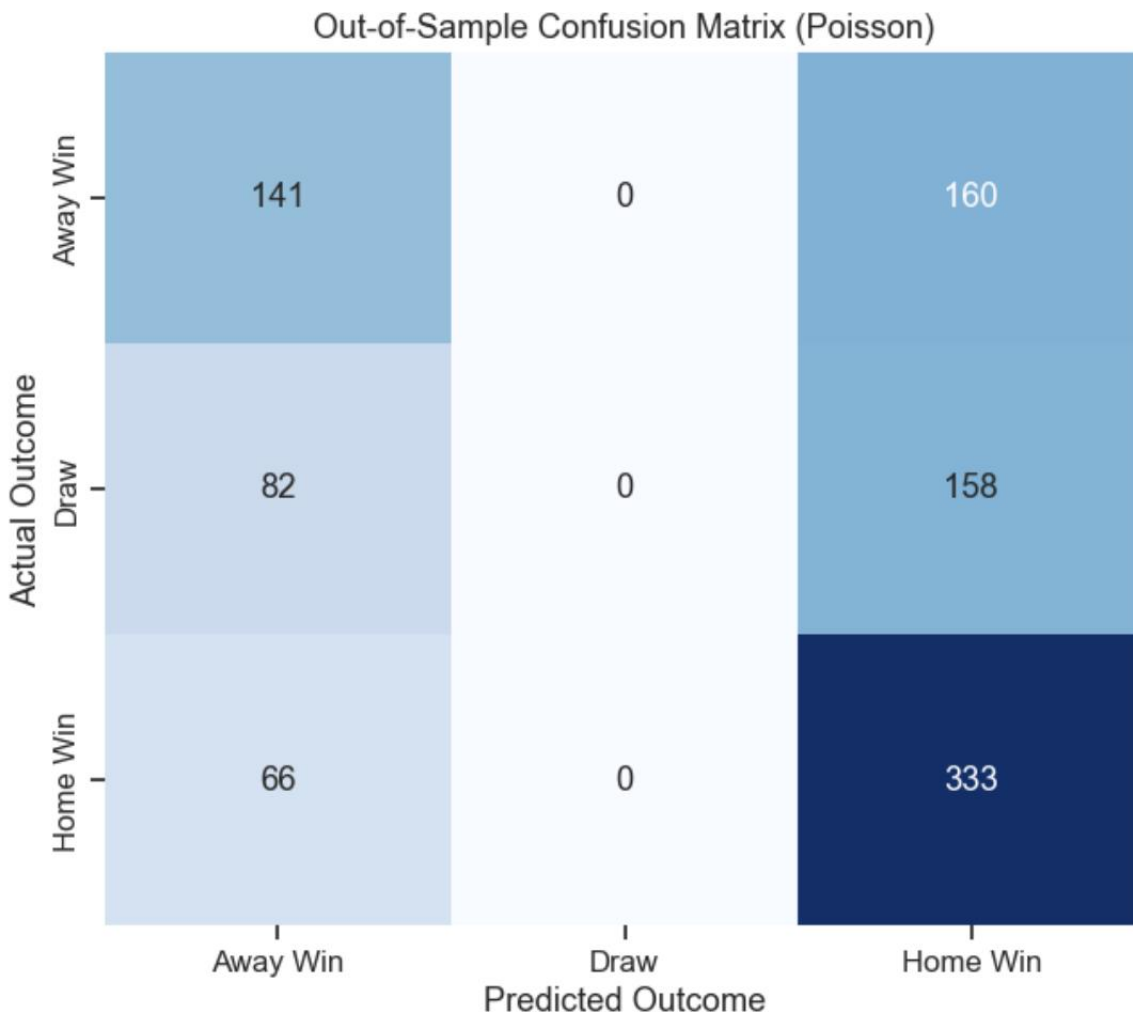


Figure 8 Confusion Matrix

9.2 Evaluations

A Confusion Matrix is a grid that places the predictions made by the computer on the bottom axis and the actual real world results on the vertical axis. Looking at this grid reveals a

fascinating quirk of the system. The model correctly predicted 333 Home Wins and 141 Away Wins. However, it predicted exactly zero Draws.

This zero does not mean the model thinks a draw is impossible. It simply means that mathematically, a draw is almost never the single highest percentage outcome in a football match. The model might calculate that a draw has a very realistic 28 percent chance of happening. However, it will usually calculate that a home win has a slightly higher 35 percent chance. Because the Confusion Matrix grid only plots the single highest percentage, the draws completely disappear from the visual chart. This demonstrates why looking only at a model's final choice is dangerous and why we must look at the raw percentage probabilities instead.

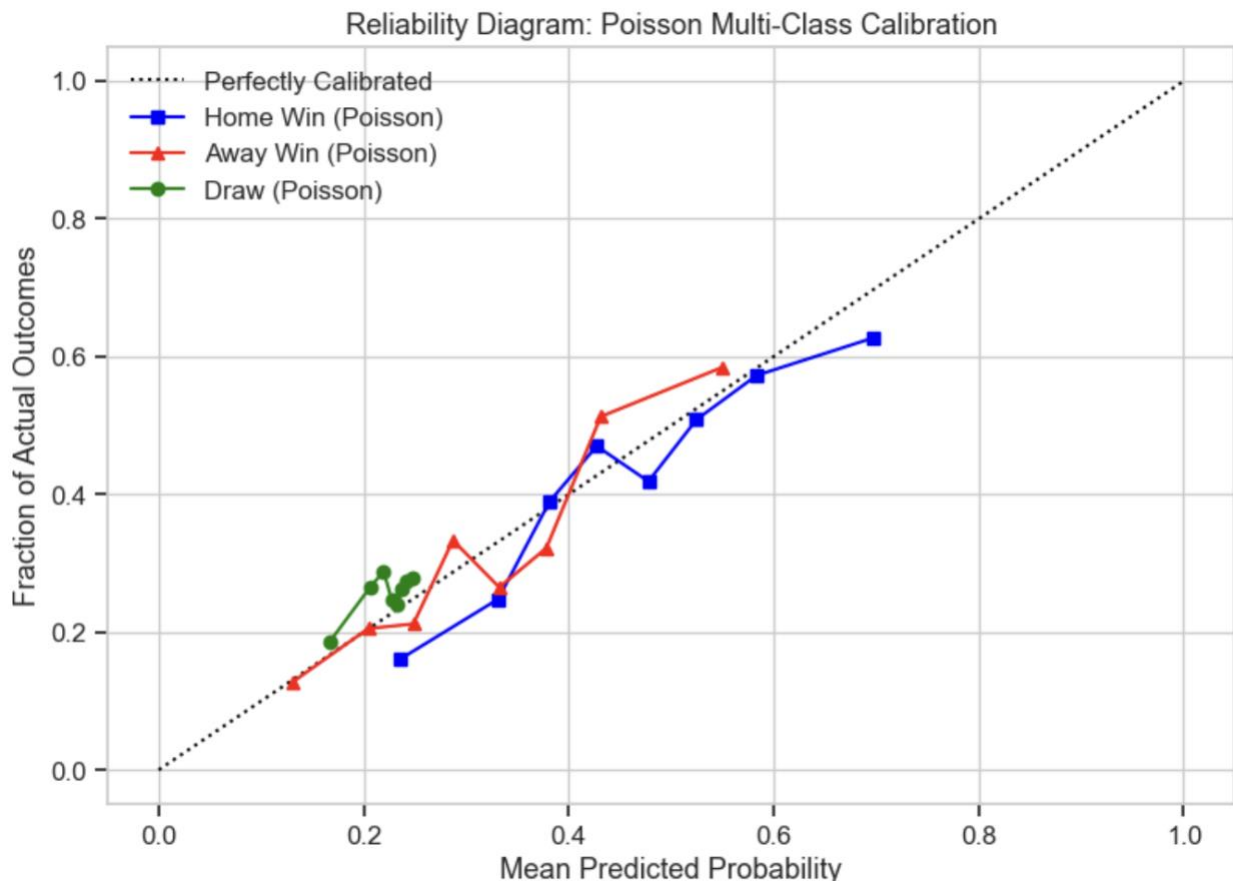


Figure 9 Reliability Diagram

To prove the raw percentages are actually accurate, the project generated a Reliability Diagram. This chart tests a simple concept. If a model says an event has a 60 percent chance of happening, that event needs to actually happen 60 out of 100 times in the real world.

The chart plots a dotted diagonal line representing perfect mathematical accuracy. The colored lines representing the model predictions for Home Wins and Away Wins track this dotted line extremely closely across the entire graph. This visual proof confirms that when the Poisson model calculates a percentage, that number is highly trustworthy and grounded in reality, even if it is not quite as precise as a billion dollar commercial betting market.

10. Practical Application

To simulate a live deployment scenario, the superior Poisson Regression framework was retrained on the entire 5-season dataset. The current rolling states (Form, Shots on Target, Travel Fatigue) were extracted dynamically to build the feature matrix for the upcoming final fixtures of the 2025/26 season (Matchday 38).

The model generated λ_h and λ_a for each fixture, traversing the Bivariate Poisson matrix to output final match probabilities, yielding the following results:

Table 6 Matchday 38 (2025/26) Predictions versus Market

Fixture (Home vs Away)	Poisson Prediction (Home / Draw / Away)	Market Odds (Home / Draw / Away)
Man City vs Aston Villa	64.8% / 19.6% / 15.5%	64.5% / 19.3% / 16.2%
Liverpool vs Brentford	54.6% / 23.1% / 22.4%	54.0% / 23.0% / 23.0%
Tottenham vs Everton	52.1% / 23.0% / 24.9%	50.8% / 24.7% / 24.5%
Brighton vs Man United	51.4% / 23.5% / 25.1%	50.5% / 23.5% / 26.0%
Burnley vs Wolves	36.0% / 24.8% / 39.1%	37.9% / 26.9% / 35.2%
Fulham vs Newcastle	35.7% / 24.7% / 39.6%	33.0% / 25.6% / 41.4%
Crystal Palace vs Arsenal	30.9% / 23.7% / 45.3%	24.6% / 24.0% / 51.4%
Nott'm Forest vs Bournemouth	36.6% / 25.5% / 37.9%	29.2% / 25.3% / 45.5%
West Ham vs Leeds	34.5% / 25.3% / 40.3%	50.7% / 25.6% / 23.7%
Sunderland vs Chelsea	42.1% / 24.4% / 33.5%	26.4% / 25.4% / 48.2%

Comparing the computer model against the financial market reveals exactly where statistical models succeed and where they fail. For high level teams with stable performance, the numbers are almost identical. The Poisson model gives Manchester City a 64.8 percent chance of winning, while the multi billion dollar betting market calculates a near perfect match at 64.5 percent. The model also calculates Liverpool and Tottenham with remarkable accuracy compared to the professional numbers. This proves the system works exactly as intended for stable conditions.

However, massive numerical disagreements exist at the bottom of the table. The Poisson model calculates a 34.5 percent chance for a West Ham home win against Leeds. The market completely disagrees and gives West Ham a massive 50.7 percent advantage. An even larger split happens in the Sunderland game. The mathematical model heavily favors Sunderland winning at home with a 42.1 percent probability. The professional market rejects this math entirely, favoring Chelsea to win on the road by a margin of 48.2 percent to 26.4 percent.

These severe disagreements happen because mathematical models are blind to human context. The Poisson model relies entirely on a strict five game historical average. It does not know if a key star player was just suspended for receiving too many yellow cards. It does not know if a team just fired their manager.

Most importantly, the model does not understand the psychology of Matchday 38. Because it is the final week of the season, teams at the bottom of the standings face relegation to a lower league. Those teams will play with extreme desperation. Conversely, teams stuck in the middle of the standings have nothing to play for and may rest their starting players. The professional betting market adjusts its percentages based on these emotional human elements. The mathematical formula only sees the numbers.

11. Conclusion

This project built a comprehensive automated system using historical data and mathematics to forecast English Premier League match results. Instead of just picking a single winner, this system calculates the exact percentage chance of a home win, a draw, and an away win. We refer to these percentage breakdowns as predictive probability distributions.

To get these calculations right, the project had to transform raw match statistics into useful indicators. A major priority during this step was preventing a common computer modeling error called data leakage. Data leakage happens when a model accidentally uses information from the future to predict the past. By strictly calculating statistics using only the matches played right up until the kickoff time of the target game, the system was forced to predict outcomes exactly as a real person would. This careful preparation allowed the system to measure complex real-world factors. These factors included a team's current momentum, the specific advantage they gain by playing at their home stadium, and the physical exhaustion caused by traveling long distances.

Testing many different types of computer algorithms revealed a very specific truth about football analytics. Highly complex models like Random Forests, XGBoost, and Neural Networks performed poorly. These advanced models are designed to find incredibly intricate patterns. Football is a low scoring sport heavily influenced by natural randomness and luck, known mathematically as aleatoric uncertainty. The complex models tried too hard to find strict rules within this random noise. This overthinking caused them to fail when trying to predict completely new, unseen matches.

On the other hand, simpler mathematical structures performed the best. The Double Poisson Regression model was the most accurate tool tested. A Poisson Regression is a statistical formula specifically built for counting independent events that happen over a set period of time. This makes it the perfect tool for predicting how many goals a team will score in 90 minutes. By treating the home team goals and away team goals as separate counting exercises, this simpler model successfully captured the true underlying strength of the teams without being fooled by the natural randomness of the sport.

While the Poisson system achieved a very strong predictive score, the commercial betting market remained slightly more accurate. This final measurement proves that historical match statistics alone do not tell the whole story. Future versions of this project will need to include new types of information beyond basic shots and goals. Adding data regarding expected goals, live player injuries, and exact lineup strengths will be necessary to completely match the accuracy of commercial odds.

APPENDIX

```
!pip3 install pandas numpy matplotlib seaborn scikit-learn statsmodels xgboost
lightgbm tensorflow scipy
import os, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from math import radians, sin, cos, sqrt, atan2
from scipy.stats import poisson
from sklearn.calibration import calibration_curve
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression, PoissonRegressor
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import log_loss, confusion_matrix
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

warnings.filterwarnings("ignore")
tf.get_logger().setLevel("ERROR")

SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

plt.style.use("seaborn-v0_8-whitegrid")
sns.set_context("notebook", font_scale=1.1)

WINDOW = 5
TSCV = TimeSeriesSplit(n_splits=5)
LABELS = [0, 1, 2] # Away=0, Draw=1, Home=2

SEASON_URLS = [
    "https://www.football-data.co.uk/mmz4281/2324/E0.csv",
```

```

    "https://www.football-data.co.uk/mmz4281/2425/E0.csv",
    "https://www.football-data.co.uk/mmz4281/2526/E0.csv",
]

COLS = [
    "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR",
    "HS", "AS", "HST", "AST", "HC", "AC", "HF", "AF",
    "B365H", "B365D", "B365A",
]

dfs = []
for url in SEASON_URLS:
    try:
        dfs.append(pd.read_csv(url, storage_options={"User-Agent": "Mozilla/5.0"}))
        print(f" Loaded {url.split('/')[-2]}")
    except Exception as e:
        print(f" Failed {url}: {e}")

df = pd.concat(dfs, ignore_index=True)[COLS].dropna().copy()
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
df = df.sort_values("Date").reset_index(drop=True)

print(f"\nTotal matches : {len(df)}")
print(f'Date range   : {df["Date"].min().date()} to {df["Date"].max().date()}')
print(f'Max nulls     : {df.isnull().sum().max()}')
stat_cols = ["FTHG", "FTAG", "HS", "AS", "HST", "AST", "HC", "AC", "HF", "AF"]

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Outcome bar chart
ax = sns.countplot(data=df, x="FTR", order=["H", "A", "D"], palette="pastel",
ax=axes[0])
axes[0].set_title("Distribution of Match Outcomes (3 Seasons)")
axes[0].set_xlabel("Full-Time Result")
axes[0].set_ylabel("Frequency")
for p in ax.patches:
    pct = f"{100 * p.get_height() / len(df):.1f}%"
    axes[0].annotate(pct, (p.get_x() + p.get_width() / 2, p.get_height()),
                      ha="center", va="bottom")

# Correlation heatmap
corr = df[stat_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
             vmin=-1, vmax=1, square=True, linewidths=0.5, ax=axes[1])
axes[1].set_title("Correlation Matrix of In-Game Statistics")

```

```

plt.tight_layout()
plt.show()

print("\n--- Continuous Variable Statistics ---")
print(df[stat_cols].describe().round(2).loc[["mean", "std", "min", "max"]])
fig, axes = plt.subplots(1, 1, figsize=(10, 8))

corr = df[stat_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True, linewidths=0.5)

plt.tight_layout()
plt.show()
home_goals, away_goals = df["FTHG"], df["FTAG"]
lam_h, lam_a = home_goals.mean(), away_goals.mean()
x_vals = np.arange(0, 9)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Home Poisson fit
sns.histplot(home_goals, bins=np.arange(0, 10) - 0.5, stat="count",
            alpha=0.6, color="steelblue", ax=axes[0])
axes[0].plot(x_vals, poisson.pmf(x_vals, lam_h) * len(df),
            "bo-", label=f"Poisson ( $\lambda=\{lam_h:.2f\}$ ")
axes[0].set_title("Home Goals: Empirical vs. Poisson")
axes[0].set_xlabel("Goals")
axes[0].legend()

# Away Poisson fit
sns.histplot(away_goals, bins=np.arange(0, 10) - 0.5, stat="count",
            alpha=0.6, color="salmon", ax=axes[1])
axes[1].plot(x_vals, poisson.pmf(x_vals, lam_a) * len(df),
            "ro-", label=f"Poisson ( $\lambda=\{lam_a:.2f\}$ ")
axes[1].set_title("Away Goals: Empirical vs. Poisson")
axes[1].set_xlabel("Goals")
axes[1].legend()

# Bivariate scoreline density
sns.kdeplot(x=df["FTHG"], y=df["FTAG"], cmap="Blues", fill=True, bw_adjust=1.2,
ax=axes[2])
axes[2].set_title("Bivariate Density of Match Scorelines")
axes[2].set_xlabel("Home Goals")
axes[2].set_ylabel("Away Goals")
axes[2].set_xticks(range(6))

```

```

axes[2].set_yticks(range(6))

plt.tight_layout()
plt.show()
df["Shot_Diff"] = df["HS"] - df["AS"]
df["Home_Win"] = (df["FTR"] == "H").astype(int)

bins = pd.qcut(df["Shot_Diff"], q=10, duplicates="drop")
prob_df = df.groupby(bins, observed=True)["Home_Win"].mean().reset_index()
prob_df["Bin_Center"] = prob_df["Shot_Diff"].apply(lambda x: x.mid).astype(float)

# Compute normalised bookmaker probabilities
df["Overround"] = 1/df["B365H"] + 1/df["B365D"] + 1/df["B365A"]
df["Prob_H"] = (1/df["B365H"]) / df["Overround"]
df["Prob_D"] = (1/df["B365D"]) / df["Overround"]
df["Prob_A"] = (1/df["B365A"]) / df["Overround"]
df["Actual_H"] = (df["FTR"] == "H").astype(int)
df["Actual_D"] = (df["FTR"] == "D").astype(int)
df["Actual_A"] = (df["FTR"] == "A").astype(int)

y = df["FTR"].map({"A": 0, "D": 1, "H": 2}).astype(int)
market_loss = log_loss(
    y, np.column_stack([df["Prob_A"], df["Prob_D"], df["Prob_H"]]), labels=LABELS
)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

axes[0].scatter(prob_df["Bin_Center"], prob_df["Home_Win"], s=100,
color="royalblue")
axes[0].axhline(0.5, color="gray", linestyle="--")
axes[0].axvline(0, color="gray", linestyle="--")
axes[0].set_title("Empirical Win Probability vs. Shot Difference")
axes[0].set_xlabel("Shot Dominance (Home Shots - Away Shots)")
axes[0].set_ylabel("P(Home Win)")

axes[1].plot([0, 1], [0, 1], "k:", label="Perfectly Calibrated")
for probs_col, actuals_col, label, color, marker in [
    ("Prob_H", "Actual_H", "Home Win (Bet365)", "blue", "s"),
    ("Prob_A", "Actual_A", "Away Win (Bet365)", "red", "^"),
    ("Prob_D", "Actual_D", "Draw (Bet365)", "green", "o"),
]:
    fp, mp = calibration_curve(df[actuals_col], df[probs_col], n_bins=10)
    axes[1].plot(mp, fp, f'{marker}-', color=color, label=label)

axes[1].set_title("Reliability Diagram: Betting Market Calibration")
axes[1].set_xlabel("Mean Predicted Probability (Bet365 Implied)")

```

```

axes[1].set_ylabel("Fraction of Actual Outcomes (Empirical)")
axes[1].legend(loc="upper left")
axes[1].set_xlim(-0.05, 1.05)
axes[1].set_ylim(-0.05, 1.05)

plt.tight_layout()
plt.show()
print(f'Bet365 market log-loss: {market_loss:.4f}')
TARGET_TEAM = "Man City" # change to any team in the dataset

tm = df[(df["HomeTeam"] == TARGET_TEAM) | (df["AwayTeam"] ==
TARGET_TEAM)].copy()
tm["Team_Scored"] = np.where(tm["HomeTeam"] == TARGET_TEAM,
tm["FTHG"], tm["FTAG"])
tm["Team_Conceded"] = np.where(tm["HomeTeam"] == TARGET_TEAM,
tm["FTAG"], tm["FTHG"])
tm["Team_Shots"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["HS"],
tm["AS"])
tm["Opp_Shots"] = np.where(tm["HomeTeam"] == TARGET_TEAM, tm["AS"],
tm["HS"])
tm = tm.reset_index(drop=True)
tm["Match_No"] = tm.index + 1
tm["Rolling_Shots"] = tm["Team_Shots"].rolling(WINDOW,
min_periods=1).mean()
tm["Rolling_Opp_Shots"] = tm["Opp_Shots"].rolling(WINDOW,
min_periods=1).mean()

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

sns.kdeplot(data=tm["Team_Scored"], fill=True, label="Goals Scored", ax=axes[0])
sns.kdeplot(data=tm["Team_Conceded"], fill=True, label="Goals Conceded",
ax=axes[0])
axes[0].set_title(f'{TARGET_TEAM}: Goals Scored vs. Conceded')
axes[0].set_xlabel("Goals per Match")
axes[0].legend()

axes[1].plot(tm["Match_No"], tm["Rolling_Shots"], label=f'{TARGET_TEAM}
Shots ({WINDOW}-game avg)')
axes[1].plot(tm["Match_No"], tm["Rolling_Opp_Shots"], label=f'Opponent Shots
({WINDOW}-game avg)')
axes[1].fill_between(tm["Match_No"],
tm["Rolling_Shots"], tm["Rolling_Opp_Shots"], alpha=0.15)
axes[1].set_title(f'{TARGET_TEAM}: Attacking vs. Defensive Volatility')
axes[1].set_xlabel("Match Number (Chronological)")
axes[1].set_ylabel("Number of Shots")
axes[1].legend()

```

<pre>plt.tight_layout() plt.show()</pre>				
#	—	Stadium	GPS	coordinates
<pre>STADIUM_COORDS = { "Arsenal": (51.5549, -0.1084), "Aston Villa": (52.5092, -1.8847), "Bournemouth": (50.7352, -1.8380), "Brentford": (51.4907, -0.2884), "Brighton": (50.8618, -0.0861), "Burnley": (53.7889, -2.2298), "Chelsea": (51.4817, -0.1910), "Crystal Palace": (51.3983, -0.0855), "Everton": (53.4388, -2.9664), "Fulham": (51.4749, -0.2217), "Ipswich": (52.0536, 1.1444), "Leeds": (53.7778, -1.5737), "Leicester": (52.6204, -1.1422), "Liverpool": (53.4308, -2.9608), "Luton": (51.8836, -0.4318), "Man City": (53.4831, -2.2004), "Man United": (53.4631, -2.2913), "Newcastle": (54.9756, -1.6218), "Nott'm Forest": (52.9399, -1.1327), "Sheffield United": (53.3703, -1.4705), "Southampton": (50.9058, -1.3911), "Sunderland": (54.9146, -1.3883), "Tottenham": (51.6043, -0.0665), "West Ham": (51.5386, -0.0163), "Wolves": (52.5904, -2.1302), }</pre>				
#	—	Helper	functions	
<pre>def haversine_miles(lat1, lon1, lat2, lon2): lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2]) a = sin((lat2 - lat1) / 2)**2 + cos(lat1) * cos(lat2) * sin((lon2 - lon1) / 2)**2 return 3958.8 * 2 * atan2(sqrt(a), sqrt(1 - a)) def build_team_log(df): """Reshape wide match data to one row per team per match.""" home = df[["Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR", "HST", "AST"]].copy()</pre>				

```

        home.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
"SOT_A"]
        home["Venue"] = "H"
        home["Points"] = home["FTR"].map({"H": 3, "D": 1, "A": 0})

        away = df[["Date", "AwayTeam", "HomeTeam", "FTAG", "FTHG", "FTR", "AST",
"HST"]].copy()
        away.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
"SOT_A"]
        away["Venue"] = "A"
        away["Points"] = away["FTR"].map({"A": 3, "D": 1, "H": 0})

        return (
            pd.concat([home, away], ignore_index=True)
            .sort_values(["Team", "Date"])
            .reset_index(drop=True)
        )

def venue_expanding_mean(group_df, venue_flag, col):
    """
    Leakage-free expanding average restricted to one venue type.
    Rows at the wrong venue become NaN; expanding().mean() skips them.
    Shift(1) ensures the current match is never included.
    """
    mask = group_df["Venue"] == venue_flag
    return (
        group_df[col]
        .where(mask)
        .expanding()
        .mean()
        .shift(1)
        .fillna(0)
    )

def compute_travel_fatigue(group_df, team_name):
    """Cumulative travel miles over the last WINDOW matches (leakage-free)."""
    if team_name not in STADIUM_COORDS:
        return pd.Series(0.0, index=group_df.index)

    hlat, hlon = STADIUM_COORDS[team_name]
    prev_lat, prev_lon = hlat, hlon
    dists = []

    for _, row in group_df.iterrows():

```

```

        clat, clon = (
            (hlat, hlon)
            if row["Venue"] == "H"
            else STADIUM_COORDS.get(row["Opponent"], (hlat, hlon))
        )
        dists.append(haversine_miles(prev_lat, prev_lon, clat, clon))
        prev_lat, prev_lon = clat, clon

    return (
        pd.Series(dists, index=group_df.index)
        .shift(1)
        .rolling(WINDOW, min_periods=1)
        .sum()
        .fillna(0)
    )

```

#	—	Build	team	log
		print("Building per-team match log...")	team_log = build_team_log(df)	
		grp = team_log.groupby("Team")		
	# — Overall rolling form (leakage-free via shift inside transform) —	print("Computing rolling form...")	team_log["Roll_GF_5"] = grp["GF"].transform(	
			lambda s: s.shift(1).rolling(WINDOW, min_periods=1).mean().fillna(0))	
		team_log["Roll_SOT_5"] = grp["SOT_F"].transform(		
			lambda s: s.shift(1).rolling(WINDOW, min_periods=1).mean().fillna(0))	
		team_log["Roll_Pts_5"] = grp["Points"].transform(		
			lambda s: s.shift(1).rolling(WINDOW, min_periods=1).sum().fillna(0))	
	# — Venue-specific expanding form (loop avoids pandas apply index issues) —	print("Computing venue-specific form...")		
		for col_out, venue_flag, col_in in [		
		("Home_GF_Form", "H", "GF"),		
		("Home_SOT_Form", "H", "SOT_F"),		
		("Away_GF_Form", "A", "GF"),		
		("Away_SOT_Form", "A", "SOT_F"),		
		]:		
		team_log[col_out] = 0.0		
		for team, g in team_log.groupby("Team"):		
		team_log.loc[g.index, col_out] = venue_expanding_mean(g,	venue_flag,	
		col_in).values		

#	—	Travel	fatigue
---	---	--------	---------

```

print("Computing travel fatigue...")
team_log["Travel_Fatigue_5"] = 0.0
for team, g in team_log.groupby("Team"):
    team_log.loc[g.index, "Travel_Fatigue_5"] = compute_travel_fatigue(g,
team).values

```

#	—	Merge	features	back	to	match	level
---	---	-------	----------	------	----	-------	-------

```

print("Merging to match level...")
FEAT_COLS = [
    "Date", "Team",
    "Roll_GF_5", "Roll_SOT_5", "Roll_Pts_5",
    "Home_GF_Form", "Home_SOT_Form",
    "Away_GF_Form", "Away_SOT_Form",
    "Travel_Fatigue_5",
]

home_feats = (
    team_log[team_log["Venue"] == "H"][FEAT_COLS]
    .rename(columns={"Team": "HomeTeam",
        **{c: f'H_{c}' for c in FEAT_COLS if c not in ("Date", "Team")}}})
)
away_feats = (
    team_log[team_log["Venue"] == "A"][FEAT_COLS]
    .rename(columns={"Team": "AwayTeam",
        **{c: f'A_{c}' for c in FEAT_COLS if c not in ("Date", "Team")}}})
)

df = df.merge(home_feats, on=["Date", "HomeTeam"], how="left")
df = df.merge(away_feats, on=["Date", "AwayTeam"], how="left")

```

#	—	Final	feature	list	&	target
---	---	-------	---------	------	---	--------

```

FEATURES = [
    "H_Roll_GF_5", "H_Roll_SOT_5", "H_Home_GF_Form",
    "H_Home_SOT_Form",
    "H_Roll_Pts_5", "H_Travel_Fatigue_5",
    "A_Roll_GF_5", "A_Roll_SOT_5", "A_Away_GF_Form",
    "A_Away_SOT_Form",
    "A_Roll_Pts_5", "A_Travel_Fatigue_5",
]

X = df[FEATURES].fillna(0)
y = df["FTR"].map({"A": 0, "D": 1, "H": 2}).astype(int)

```

```

N_FEAT = X.shape[1] # 12

print(f"\nFeature matrix : {X.shape}")
print(f"Target                                counts")
:\n{y.value_counts().sort_index().rename({0:'Away',1:'Draw',2:'Home'})})")
print("\n--- Feature Summary ---")
print(X.describe().round(3).loc[["mean", "std", "min", "max"]])
# Note: solver='saga' replaces 'lbfgs' for LogisticRegression — 'lbfgs' was
# producing hundreds of divide-by-zero/overflow warnings from sklearn's matmul
# on this dataset. 'saga' is numerically more stable for multiclass problems.
#
# Note on LSTM architecture: input_shape=(1, N_FEAT) treats each match as a
# single timestep. This replicates the original design. True sequential
# modelling would pass a sliding window of prior matches as the time dimension.

BASE_MODELS = {
    "1. Logistic Regression": LogisticRegression(
        solver="saga", max_iter=2000, random_state=SEED),
    "2. Random Forest": RandomForestClassifier(random_state=SEED),
    "3. Gradient Boosting": GradientBoostingClassifier(random_state=SEED),
    "4. XGBoost": XGBClassifier(
        objective="multi:softprob", eval_metric="mlogloss",
        random_state=SEED, verbosity=0),
    "5. LightGBM": LGBMClassifier(random_state=SEED, verbose=-1),
    "6. Neural Network (MLP)": MLPClassifier(
        solver="adam", max_iter=500, early_stopping=True, random_state=SEED),
}

PARAM_GRIDS = {
    "1. Logistic Regression": {
        "model__C": [0.01, 0.1, 1.0],
        "model__penalty": ["l2"],
    },
    "2. Random Forest": {
        "model__n_estimators": [100],
        "model__max_depth": [5, 10, None],
    },
    "3. Gradient Boosting": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__max_depth": [3, 5],
    },
    "4. XGBoost": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__max_depth": [3, 5],
    },
}

```

```

    },
    "5. LightGBM": {
        "model__n_estimators": [100],
        "model__learning_rate": [0.05, 0.1],
        "model__num_leaves": [31, 63],
    },
    "6. Neural Network (MLP)": {
        "model__hidden_layer_sizes": [(32,), (64, 32)],
        "model__alpha": [0.001, 0.01],
    },
}

```

```

def build_lstm(n_features):
    """Build and compile the LSTM used throughout this notebook."""
    model = Sequential([
        LSTM(32, input_shape=(1, n_features), activation="relu"),
        Dropout(0.2),
        Dense(16, activation="relu"),
        Dense(3, activation="softmax"),
    ])
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")
    return model

```

```
print("Model definitions ready.")
```

#	—	Hyperparameter	tuning
---	---	----------------	--------

```

print("--- Hyperparameter Tuning ---\n")
tuned_models = {}
for name, base_model in BASE_MODELS.items():
    pipe = Pipeline([("scaler", StandardScaler()), ("model", base_model)])
    grid = GridSearchCV(
        estimator=pipe, param_grid=PARAM_GRIDS[name],
        cv=TSCV, scoring="neg_log_loss", n_jobs=-1, refit=True,
    )
    grid.fit(X, y)
    tuned_models[name] = grid.best_estimator_
    print(f'[{name}] best log-loss: {-grid.best_score_:4f}')

```

#	—	Walk-forward	evaluation
---	---	--------------	------------

```

print("\n--- Walk-Forward Evaluation ---\n")
results = {}
poisson_y_true, poisson_y_prob = [], [] # Saved for Poisson confusion matrix &
calibration

```

```

# 1. Standard sklearn models
for name, pipe in tuned_models.items():
    f_logloss, f_acc, f_f1 = [], [], []
    for train_idx, test_idx in TSCV.split(X):
        pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
        probs = pipe.predict_proba(X.iloc[test_idx])
        preds = np.argmax(probs, axis=1)
        y_test = y.iloc[test_idx]

        f_logloss.append(log_loss(y_test, probs, labels=LABELS))
        f_acc.append(accuracy_score(y_test, preds))
        f_f1.append(f1_score(y_test, preds, average='macro'))

    results[name] = {'Log-Loss': np.mean(f_logloss), 'Accuracy': np.mean(f_acc), 'F1-
Macro': np.mean(f_f1)}

# 2. Poisson regression
f_logloss, f_acc, f_f1 = [], [], []
for train_idx, test_idx in TSCV.split(X):
    sc = StandardScaler()
    X_tr, X_te = sc.fit_transform(X.iloc[train_idx]), sc.transform(X.iloc[test_idx])

    pr_h = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_tr,
df["FTHG"].iloc[train_idx])
    pr_a = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_tr,
df["FTAG"].iloc[train_idx])

    probs = np.zeros((len(test_idx), 3))
    for i, (lh, la) in enumerate(zip(pr_h.predict(X_te), pr_a.predict(X_te))):
        lh, la = max(float(lh), 1e-6), max(float(la), 1e-6)
        ph, pd_tie, pa = 0.0, 0.0, 0.0
        for h in range(10):
            for a in range(10):
                ps = poisson.pmf(h, lh) * poisson.pmf(a, la)
                if h > a: ph += ps
                elif h == a: pd_tie += ps
                else: pa += ps
            tot = ph + pd_tie + pa
            probs[i] = [pa / tot, pd_tie / tot, ph / tot]

    preds = np.argmax(probs, axis=1)
    y_test = y.iloc[test_idx]

    f_logloss.append(log_loss(y_test, probs, labels=LABELS))
    f_acc.append(accuracy_score(y_test, preds))

```

```

    f_f1.append(f1_score(y_test, preds, average='macro'))

    poisson_y_true.extend(y_test.tolist())
    poisson_y_prob.extend(probs.tolist())

    results["7. Poisson Regression"] = {'Log-Loss': np.mean(f_logloss), 'Accuracy':
np.mean(f_acc), 'F1-Macro': np.mean(f_f1)}

# 3. RNN (LSTM)
f_logloss, f_acc, f_f1 = [], [], []
for train_idx, test_idx in TSCV.split(X):
    sc = StandardScaler()
    X_tr = sc.fit_transform(X.iloc[train_idx]).reshape(-1, 1, N_FEAT)
    X_te = sc.transform(X.iloc[test_idx]).reshape(-1, 1, N_FEAT)

    lstm = build_lstm(N_FEAT)
    lstm.fit(X_tr, y.iloc[train_idx], epochs=40, batch_size=16, verbose=0)

    probs = lstm.predict(X_te, verbose=0)
    preds = np.argmax(probs, axis=1)
    y_test = y.iloc[test_idx]

    f_logloss.append(log_loss(y_test, probs, labels=LABELS))
    f_acc.append(accuracy_score(y_test, preds))
    f_f1.append(f1_score(y_test, preds, average='macro'))

    results["8. RNN (LSTM)"] = {'Log-Loss': np.mean(f_logloss), 'Accuracy':
np.mean(f_acc), 'F1-Macro': np.mean(f_f1)}

# 4. Market Baseline
m_preds = np.argmax(np.column_stack([df["Prob_A"], df["Prob_D"],
df["Prob_H"]]), axis=1)
results["Bet365 (Baseline)"] = {
    'Log-Loss': market_loss,
    'Accuracy': accuracy_score(y, m_preds),
    'F1-Macro': f1_score(y, m_preds, average='macro')
}

#          —————          Print          ranking
print(f"\n{'Model':<30} | {'Log-Loss ↓':<10} | {'Accuracy ↑':<10} | {'F1-Macro
↑':<10}")
print("-" * 68)
# Sort models by Log-Loss (lowest is best)
for name, metrics in sorted(results.items(), key=lambda x: x[1]['Log-Loss']):
    marker = " <--" if "Bet365" in name else ""

```

```

        print(f' {name:<30} | {metrics['Log-Loss']:.4f} | {metrics['Accuracy']:.4f} | {metrics['F1-Macro']:.4f} {marker}')
    sorted_res = dict(sorted(results.items(), key=lambda x: x[1]['Log-Loss']))
    colors = ["#ff9999" if "Bet365" in n else "#66b3ff" for n in sorted_res]

    plt.figure(figsize=(10, 6))
    log_losses = [metrics["Log-Loss"] for metrics in sorted_res.values()]
    bars = plt.barh(list(sorted_res.keys()), log_losses,
                    color=colors, edgecolor="white")
    plt.axvline(market_loss, color="red", linestyle="--", alpha=0.7,
                label=f'Bet365 baseline ({market_loss:.4f})')
    for bar in bars:
        plt.text(bar.get_width() - 0.005,
                 bar.get_y() + bar.get_height() / 2,
                 f'{bar.get_width():.4f}',
                 va="center", ha="right", color="black", fontweight="bold", fontsize=9)
    plt.xlabel("Cross-Validated Log-Loss (lower is better)")
    plt.title("Model Performance vs. Betting Market")
    plt.legend()
    plt.tight_layout()
    plt.show()
#          — Poisson Regression Coefficients

print("==== 7. Poisson Regression — Feature Coefficients =====")
sc_full = StandardScaler()
X_sc = sc_full.fit_transform(X)

pr_h_full = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_sc, df["FTHG"])
pr_a_full = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_sc, df["FTAG"])

coef_df = pd.DataFrame({
    "Feature": FEATURES,
    "Home Goals ( $\lambda$ ) Coef": pr_h_full.coef_,
    "Away Goals ( $\lambda$ ) Coef": pr_a_full.coef_
})

# We'll sort by the absolute impact on Home Goals to see what drives the match
coef_df["Abs_Home_Impact"] = coef_df["Home Goals ( $\lambda$ ) Coef"].abs()
coef_df = coef_df.sort_values("Abs_Home_Impact",
ascending=False).drop(columns="Abs_Home_Impact")

print(coef_df.to_string(index=False))

# Plot the coefficients for visual comparison
coef_df.set_index("Feature").plot(kind="barh",
                                figsize=(10, 6),
                                colormap="coolwarm")

```

```

plt.title("Feature Impact on Expected Goals ( $\lambda$ ) - Poisson Regression")
plt.xlabel("Coefficient Weight")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
poisson_y_true_arr = np.array(poisson_y_true)
poisson_y_prob_arr = np.array(poisson_y_prob)
poisson_preds = np.argmax(poisson_y_prob_arr, axis=1)
class_labels = ["Away Win", "Draw", "Home Win"]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Confusion matrix
cm = confusion_matrix(poisson_y_true_arr, poisson_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_labels, yticklabels=class_labels, ax=axes[0])
axes[0].set_title("Out-of-Sample Confusion Matrix (Poisson)")
axes[0].set_xlabel("Predicted Outcome")
axes[0].set_ylabel("Actual Outcome")

# Calibration curve
axes[1].plot([0, 1], [0, 1], "k:", label="Perfectly Calibrated")
for cls_idx, label, color, marker in [
    (2, "Home Win (Poisson)", "blue", "s"),
    (0, "Away Win (Poisson)", "red", "^"),
    (1, "Draw (Poisson)", "green", "o"),
]:
    actual = (poisson_y_true_arr == cls_idx).astype(int)
    fp, mp = calibration_curve(actual, poisson_y_prob_arr[:, cls_idx],
                             n_bins=8, strategy="quantile")
    axes[1].plot(mp, fp, f'{marker}-', color=color, label=label)

axes[1].set_title("Reliability Diagram: Poisson Multi-Class Calibration")
axes[1].set_xlabel("Mean Predicted Probability")
axes[1].set_ylabel("Fraction of Actual Outcomes")
axes[1].legend(loc="upper left")
axes[1].set_xlim(-0.05, 1.05)
axes[1].set_ylim(-0.05, 1.05)

plt.tight_layout()
plt.show()
FINAL_WEEK = [
    {"HomeTeam": "Man City", "AwayTeam": "Aston Villa"},
    {"HomeTeam": "Brighton", "AwayTeam": "Man United"},
    {"HomeTeam": "Fulham", "AwayTeam": "Newcastle"},
    {"HomeTeam": "Tottenham", "AwayTeam": "Everton"},

```

```

        {"HomeTeam": "Liverpool",    "AwayTeam": "Brentford"},
        {"HomeTeam": "Burnley",      "AwayTeam": "Wolves"},
        {"HomeTeam": "Nott'm Forest", "AwayTeam": "Bournemouth"},
        {"HomeTeam": "West Ham",     "AwayTeam": "Leeds"},
        {"HomeTeam": "Sunderland",   "AwayTeam": "Chelsea"},
        {"HomeTeam": "Crystal Palace", "AwayTeam": "Arsenal"},
    ]

def get_latest_features(team: str, is_home: bool) -> dict:
    """Return the most recent rolling features for a team from team_log."""
    hist = team_log[team_log["Team"] == team].tail(1)
    p = "H_" if is_home else "A_"
    vp = "Home_" if is_home else "Away_"
    zeros = {
        f"{p}Roll_GF_5": 0.0, f"{p}Roll_SOT_5": 0.0, f"{p}Roll_Pts_5": 0.0,
        f"{p}{vp}GF_Form": 0.0, f"{p}{vp}SOT_Form": 0.0,
        f"{p}Travel_Fatigue_5": 0.0,
    }
    if hist.empty:
        return zeros
    r = hist.iloc[0]
    return {
        f"{p}Roll_GF_5": r["Roll_GF_5"],
        f"{p}Roll_SOT_5": r["Roll_SOT_5"],
        f"{p}Roll_Pts_5": r["Roll_Pts_5"],
        f"{p}{vp}GF_Form": r[f"{vp}GF_Form"],
        f"{p}{vp}SOT_Form": r[f"{vp}SOT_Form"],
        f"{p}Travel_Fatigue_5": r["Travel_Fatigue_5"],
    }

# Train final Poisson models on all available data
print("Training deployment Poisson Regression on full dataset..")
sc_deploy = StandardScaler()
X_all_sc = sc_deploy.fit_transform(X)

pr_h_deploy = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_all_sc,
df["FTHG"])
pr_a_deploy = PoissonRegressor(alpha=0.1, max_iter=1000).fit(X_all_sc,
df["FTAG"])

# Build live feature matrix (column order must match FEATURES)
X_live = pd.DataFrame([
    **get_latest_features(m["HomeTeam"], True),
    **get_latest_features(m["AwayTeam"], False)}
    for m in FINAL_WEEK

```

```

])[FEATURES].fillna(0)

X_live_sc = sc_deploy.transform(X_live)

# Calculate bivariate Poisson probabilities for live matches
live_preds = []
for lh, la in zip(pr_h_deploy.predict(X_live_sc), pr_a_deploy.predict(X_live_sc)):
    lh, la = max(float(lh), 1e-6), max(float(la), 1e-6)
    ph, pd_tie, pa = 0.0, 0.0, 0.0
    for h in range(10):
        for a in range(10):
            ps = poisson.pmf(h, lh) * poisson.pmf(a, la)
            if h > a: ph += ps
            elif h == a: pd_tie += ps
            else: pa += ps
    tot = ph + pd_tie + pa
    live_preds.append((pa / tot, pd_tie / tot, ph / tot))

# Print predictions
print(f"\n--- Matchday 38 (2025/26) Predictions — Poisson Regression ---\n")
header = f'{'Fixture':<35} | {'Home %':>7} | {'Draw %':>7} | {'Away %':>7} | {'H
Odds':>7} | {'D Odds':>7} | {'A Odds':>7}"
print(header)
print("-" * len(header))
for m, (pa, pd_tie, ph) in zip(FINAL_WEEK, live_preds):
    # Clamp to avoid division by zero in implied odds
    ph, pd_tie, pa = max(float(ph), 1e-4), max(float(pd_tie), 1e-4), max(float(pa), 1e-4)
    fixture = f'{m['HomeTeam']} vs {m['AwayTeam']}'
    print(
        f'{'fixture':<35} | {ph*100:>6.1f}% | {pd_tie*100:>6.1f}% | {pa*100:>6.1f}% "
        f'| {1/ph:>7.2f} | {1/pd_tie:>7.2f} | {1/pa:>7.2f}"
    )

```